

RadYOLO 중간보고서

mmWave 레이더 + 웹캠 기반
실내 3D 씬 재구성 및 실시간 객체 추적

GitHub <https://github.com/superloser030/RadYOLO>

작성일 2026년 6월 26일

0 Contents

1	연구 목표	1
1.1	세부 목표	1
2	시스템 구성	1
2.1	하드웨어	1
2.2	네트워크 구성	1
2.3	소프트웨어 스택	1
2.4	디렉토리 구조	1
3	전체 파이프라인	2
3.1	실시간 전송 (sender.py)	2
3.2	수신 및 저장 (receiver.py)	3
3.3	배경 처리 파이프라인	3
3.4	MATLAB 레이더 신호처리 (cfar_detect.m)	4
3.5	DA3 Depth Metric 보정 (depth_calibration.py)	4
	3.5.1 보정 모델	4
	3.5.2 2-앵커 직접 풀이	4
	3.5.3 최근 보정 결과	4
3.6	3D 뷰어 (viewer.html)	5
	3.6.1 포인트 클라우드 투영	5
	3.6.2 객체 오버레이	5
4	주요 트러블슈팅	5
4.1	DCA1000 자동 종료 문제	5
5	현재 구현 상태	5
6	향후 계획	5
6.1	DA3 Depth 보정 고도화	5
6.2	실시간 객체 위치 뷰어 표시	6
6.3	사람 3D 메시 실시간 렌더링	6
6.4	동적 배경 자동 갱신	6

1 연구 목표

본 연구는 TI AWR1642BOOST mmWave 레이더와 웹캠을 융합하여 실내 환경의 3D 씬을 자동으로 재구성하고, 실시간으로 다중 인물을 감지 및 추적하는 시스템을 개발하는 것을 목표로 한다.

1.1 세부 목표

1. 연구실(송신) → 기숙사(수신) 실시간 레이더/웹캠 데이터 전송 파이프라인 구축
2. Depth Anything V3(DA3) + 레이더 거리 측정을 융합한 metric 깊이 추정
3. Trellis 3D 모델 생성 및 GigaPose 포즈 추정을 통한 객체 3D 배치
4. Three.js 기반 실시간 3D 뷰어에서 배경 포인트 클라우드 및 객체 메시 표시

2 시스템 구성

2.1 하드웨어

구성 요소	모델	역할
레이더	TI AWR1642BOOST	mmWave IQ 데이터 수집 (TX×2, RX×4)
캡처 보드	TI DCA1000EVM	LVDS → Ethernet UDP 변환, raw 저장
카메라	USB 웹캠	RGB 영상 캡처 (1920×1080, 10fps)
송신 PC	연구실 노트북	DCA1000 제어 + 데이터 전송
수신 PC	기숙사 데스크톱	처리, 3D 재구성, 뷰어 실행

Table 1: 하드웨어 구성

2.2 네트워크 구성

```
AWR1642BOOST
|-- UART (COM3, 115200 baud) --> send_config.py (.cfg 전송)
|-- LVDS (60pin Samtec) --> DCA1000EVM
    |-- Ethernet UDP (192.168.33.30:4098)
        --> 연구실노트북 (fordev-kjh, 100.80.189.85)
            |-- Tailscale VPN / UDP
                --> 기숙사데스크톱 (forlife-kjh, 100.126.220.19)
                    RADAR_PORT = 5006
                    WEBCAM_PORT = 5007
```

대역폭: 핫스팟 환경 평균 약 24.9 Mbps (Tailscale 기준)

2.3 소프트웨어 스택

2.4 디렉토리 구조

구성 요소	버전/모델	용도
Python	3.11 (conda)	전체 파이프라인
MATLAB	R2026a	CFAR 레이더 신호처리
ComfyUI	–	DA3 깊이 추정, ESRGAN 업스케일
Depth Anything V3	da3_large	단안 깊이 추정
YOLOv8x-seg	Ultralytics	객체 감지 및 세그멘테이션
Trellis	fal.ai API	단일 이미지 → 3D 메시(GLB) 생성
GigaPose	–	3D 객체 6DoF 포즈 추정
Three.js	0.160.0	실시간 3D 웹 뷰어

Table 2: 소프트웨어 스택

```

RadYOLO/
|-- main.py                                # 전체파이프라인진입점
|-- config/
|   |-- camera.json                       # 카메라내부파라미터 (fx, fy, cx, cy)
|   |-- sender.json                       # 송신설정 (fps, quality, 레이더주기 )
|   |-- receiver.json                     # 수신설정
|-- src/
|   |-- transmission/
|   |   |-- sender.py                     # 연구실: 웹캠레이더+ UDP 송신
|   |   |-- receiver.py                   # 기숙사: 수신 + binary 저장 + 타임스탬프
|   |-- background/
|   |   |-- bg_select.py                  # 최적배경프레임선택
|   |   |-- upscale.py                    # ESRGAN x4 업스케일
|   |   |-- depth.py                      # ComfyUI DA3 깊이추정
|   |   |-- depth_calibration.py          # 레이더기반 metric 보정
|   |   |-- yolo_mask.py                  # YOLOv8 세그멘테이션마스크
|   |-- objects/
|   |   |-- obj_crop.py                   # 객체크롭
|   |   |-- trellis_gen.py                # Trellis 3D 모델생성
|   |   |-- pose_estimator.py             # GigaPose 포즈추정
|   |   |-- flux_kontext.py               # Flux 인페인팅
|   |-- utils/archive.py                  # 세션데이터아카이브
|   |-- viewer/viewer.html                # Three.js 실시간 3D 뷰어
|-- matlab/
|   |-- cfar_detect.m                     # CFAR 검출 -> targets.json
|   |-- process_radar.m
|-- workflows/                             # ComfyUI JSON 워크플로우
|   |-- da3_depth.json
|   |-- upscale_esrgan.json
|   |-- flux_kontext.json

```

3 전체 파이프라인

3.1 실시간 전송 (sender.py)

연구실 노트북에서 실행되며, 레이더 IQ 데이터와 웹캠 영상을 동시에 전송한다.

1. DCA1000EVM CLI로 FPGA 초기화 및 녹화 시작
2. send_config.py: pyserial로 COM3에 .cfg 파일 전송 (sensorStart 포함)

3. radar_forward(): DCA1000에서 수신한 UDP 패킷을 기속사로 포워딩
4. webcam_send(): OpenCV 웹캠 프레임을 JPEG 압축 후 청크 단위로 전송
5. DCA1000은 약 70,000 패킷 후 자동 종료되므로 해당 시점에 자동 재시작

3.2 수신 및 저장 (receiver.py)

1. 레이더 패킷 수신 → data/radar/iqData_*.bin 저장
2. 각 프레임마다 iqData_timestamps.csv에 frame_idx, ts_ms 기록
3. 타임스탬프: ms-since-midnight 방식으로 레이더/웹캠 동기화
4. 웹캠 프레임을 frame_queue에 적재 → bg_select.py에서 소비

3.3 배경 처리 파이프라인

```

Step 1  bg_select.py초간
        10 웹캠프레임수신 , 가장선명한프레임선택
        --> background_raw.jpg, background_ts.txt

Step 2  upscale.py
        ESRGAN x4 업스케일 (ComfyUI 워크플로우)
        --> background.jpg

Step 3  depth.py
        DA3 Large 모델로단안깊이추정정규화
        : Standard (near=bright, 밝을수록가까운거리 )
        --> depth.png

Step 3.5 depth_calibration.py레이더
        targets.에서json 타임스탬프매칭 (+-1000ms)
        2-anchor log-linear 모델 fitting
        --> depth.png 보정됨(), depth_calib.json

Step 4  yolo_mask.py
        YOLOv8x-로seg 사람영역마스킹
        --> mask.png

Step 5  obj_crop.py감지된객체별크롭이미지저장

        --> data/objects/<cls>/cutout.jpg

Step 5.3 trellis_gen.py
        fal.ai Trellis 로API 단일이미지 -> 3D 메시생성
        --> model_trellis.glb

Step 5.5 pose_estimator.py로
        GigaPose 6DoF 포즈추정
        --> pose.json, manifest.json

Live    live_update_loop()
        YOLO 위치갱신초 (0.3), GigaPose 회전갱신연속 ()
        --> live.json, pose.json 실시간( 갱신)

```

3.4 MATLAB 레이더 신호처리 (cfar_detect.m)

MATLAB에서 별도 실행하며, receiver.py가 저장한 IQ 데이터를 처리한다.

1. iqData_RecordingParameters.mat에서 레이더 파라미터 로드
2. TX1 홀수 chirp만 사용, 4개 RX에 대해 Range-Doppler 맵 계산
3. phased.CFARDetector2D로 2D CFAR 검출
4. 4-element steering vector FFT로 방위각 추정
5. 결과 저장: targets.json

3.5 DA3 Depth Metric 보정 (depth_calibration.py)

DA3가 출력하는 깊이 맵은 상대적 깊이(0~1 정규화)만 제공하므로, 레이더 거리 측정값을 이용해 실제 미터 단위로 보정한다.

3.5.1 보정 모델

레이더 range 값 R 과 DA3 픽셀값 $D \in [0, 1]$ 사이의 관계를 로그-선형 모델로 근사한다:

$$\log R = a \cdot D + b \iff R = e^{aD+b} \quad (1)$$

3.5.2 2-앵커 직접 풀이

- 앵커 1: $D_{99\%}$ (가장 밝은 픽셀, near) $\leftrightarrow R_{1\%}$ (가장 가까운 레이더 타겟)
- 앵커 2: $D_{1\%}$ (가장 어두운 픽셀, far) $\leftrightarrow R_{99\%}$ (가장 먼 레이더 타겟)

두 조건으로 2×2 연립방정식을 구성하여 a, b 를 직접 풀이한다.

3.5.3 최근 보정 결과

파라미터	값	의미
a	-2.301	기울기
b	2.350	절편
$D = 1$ (근거리)	1.05 m	최근접 추정 거리
$D = 0$ (원거리)	10.49 m	최원 추정 거리

Table 3: depth_calibration.py 보정 결과 (2026-06-26)

보정된 깊이 맵은 log-scale 재정규화를 적용하여 근거리와 원거리의 시각적 비중을 균등하게 만든다:

$$D_{\text{norm}} = 1 - \frac{\ln R - \ln R_{\min}}{\ln R_{\max} - \ln R_{\min}} \quad (2)$$

3.6 3D 뷰어 (viewer.html)

Three.js 기반 웹 뷰어로, `python main.py --viewer-only` 실행 시 `http://localhost:8000/src/viewer/viewer.html` 에서 접속한다.

3.6.1 포인트 클라우드 투영

깊이 맵 픽셀 (p_x, p_y, d) 를 3D 공간으로 투영할 때 올바른 원근 투영 (perspective projection) 을 적용한다:

$$d_{\text{cam}} = e^{a \cdot d + b} \quad (\text{depth_calib.json 로드 시}) \quad (3)$$

$$X = \frac{p_x - c_x}{f_x} \cdot d_{\text{cam}} \quad (4)$$

$$Y = -\frac{p_y - c_y}{f_y} \cdot d_{\text{cam}} \quad (5)$$

$$Z = -d_{\text{cam}} \quad (6)$$

3.6.2 객체 오버레이

GigaPose `pose.json` 에서 회전 행렬 R 과 이동 벡터 t 를 읽어 카메라 좌표계 $\rightarrow$ Three.js 좌표계로 변환 후 GLB 메시를 씬에 배치한다:

$$F = \text{diag}(1, -1, -1), \quad R_{\text{view}} = F R_{\text{cam}} F^T \quad (7)$$

4 주요 트러블슈팅

4.1 DCA1000 자동 종료 문제

증상: DCA1000EVM이 약 7,000 패킷 수신 후 자동으로 녹화를 종료함.

원인: DCA1000 하드웨어의 내부 버퍼 한계로 인해 장시간 연속 녹화가 불가능.

해결: `sender.py` 에서 시퀀스 번호가 `RESTART_AT_SEQ = 70000` 에 도달하면 `stop_record` $\rightarrow$ `start_record` 를 자동 실행하여 세션을 재시작. 재시작 중 수집되지 않는 프레임은 타임스탬프 기반으로 식별 가능.

5 현재 구현 상태

6 향후 계획

6.1 DA3 Depth 보정 고도화

현재 보정은 레이더 range 분포와 DA3 픽셀 분포를 2개의 앵커 포인트로 매칭하는 단순 log-linear 모델을 사용한다. 향후 레이더 방위각 정보와 카메라 투영을 결합하여 픽셀 단위의 정밀 보정을 구현하고, 보정 신뢰도를 정량적으로 평가할 예정이다.

기능	내용	상태
실시간 전송	레이더/웹캠 UDP 전송, 타임스탬프 동기화	완료
배경 선택	10초 수집 후 최적 프레임 선택	완료
ESRGAN 업스케일	배경 이미지 해상도 향상	완료
DA3 깊이 추정	단안 깊이 맵 생성	완료
Depth Metric 보정	레이더 기반 log-linear 보정	완료
CFAR 레이더 검출	MATLAB, targets.json 출력	완료
YOLO 마스킹	사람 영역 세그멘테이션	완료
Trellis 3D 생성	단일 이미지 → GLB 메시	완료
GigaPose 포즈 추정	6DoF 포즈 추정	완료
3D 뷰어	포인트 클라우드 + GLB 오버레이	완료

Table 4: 구현 현황

6.2 실시간 객체 위치 뷰어 표시

YOLO로 감지된 객체의 2D 바운딩박스 중심을 깊이 맵으로 역투영하여 3D 공간 좌표를 계산하고, 뷰어에 실시간으로 표시한다. 레이더 타겟과 YOLO 바운딩박스를 매칭하여 ID 기반 다중 인물 추적을 구현할 예정이다.

6.3 사람 3D 메시 실시간 렌더링

감지된 사람 영역을 Trellis 또는 별도 인체 메시 모델을 이용해 3D 메시로 생성하고 뷰어에 실시간으로 렌더링한다. 포즈 추정 결과를 반영하여 방향까지 표현하는 것을 목표로 한다.

6.4 동적 배경 자동 갱신

카메라 시야 내에 이전에 보이지 않았던 배경 영역이 새롭게 노출되면, 해당 영역을 자동으로 감지하고 Flux 인페인팅으로 배경 맵을 갱신한다. 사람이 이동하면서 가려졌던 벽이나 바닥이 드러날 때 자연스럽게 3D 씬이 보완되는 효과를 목표로 한다.